1 Setup do projeto

1.1 Criar o projeto

```
$ mkdir meu-projeto && cd meu-projeto
$ npm init -y
```

1.2 package.json

ES Modules e scripts.

```
$ npm pkg set type=module
$ npm pkg set scripts.start="node server.js" scripts.dev="node --watch server.js"
```

1.3 Instalar os pacotes

Fixar @6 no Prisma: sem versão vem a 8.0.0-rc e o **`init`** falha.

```
$ npm install express ejs method-override @prisma/client@6
$ npm install --save-dev prisma@6
```

Testar: existe **`node_modules/`** e o **`package.json`** lista as dependências.

2 Pastas e arquivos

2.1 src/

Criar **`config`**, **`controllers`**, **`services`**, **`repositories`**, **`routes`** e **`views`**.

2.2 public/

Criar **`css/`** e **`js/`**.

2.3 Raiz do projeto

Criar **`server.js`** (extensão **.js**), **`.env`**, **`.gitignore`** e **`README.md`**.

3 server.js e app.js

3.1 server.js

Importar o app e chamar **`app.listen`** na porta 3000. Sem log.

Abrir: Cap. 3 §3.5.3 — ver o server.js (import do app e listen)
https://leofernandesmo.github.io/pweb2-book/chapters/03-express/#353-separacao-entre-appjs-e-serverjs

3.2 src/app.js

Express com EJS (views em **`src/views`**), **`urlencoded`**, **`public/`** estático e **`export default app`**. **Sem rotas ainda.** Criar o **`__dirname`** com **`fileURLToPath(import.meta.url)`** e usar **`.js`** nos imports.

Abrir: Cap. 6 §6.3.2 — ver o app.js com EJS, urlencoded e static
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#632-configuracao-do-express

4 Primeira tela em EJS (hello world)

4.1 src/routes/index.routes.js

Router com **`GET /`** renderizando o **`index`** com título e mensagem.

Abrir: Cap. 6 §6.3.4 — ver a rota com res.render e o index.ejs
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#634-renderizando-a-primeira-view

4.2 src/app.js

Importar o router e adicionar o **`app.use`** antes do **`export default app`**. Sem isso: **`Cannot GET /`**.

```
import router from './routes/index.routes.js';
app.use('/', router);
```

4.3 src/views/index.ejs

CSS e JS com caminho absoluto, **sem `public/`**. Este esqueleto vale para toda view nova.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Início</title>
  <link rel="stylesheet" href="/css/style.css">
  <script src="/js/main.js" defer></script>
</head>
<body>
  <h1><%= mensagem %></h1>
</body>
</html>
```

4.4 public/css/style.css

Reset (margem e padding zero) e **`body`** com **`background: #88e590`**.

Abrir: Cap. 6 §6.8.3 — ver o bloco de CSS (reset e body)
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#683-css-minimo-de-suporte

4.5 Rodar

```
$ npm run dev
```

Testar: abrir **`http://localhost:3000`**: mensagem e fundo verde; **`/css/style.css`** e **`/js/main.js`** abrem (com **`/public/...`** dá 404).

5 Prisma + SQLite

5.1 Inicializar

```
$ npx prisma init --datasource-provider sqlite
```

5.2 prisma/schema.prisma

Trocar o gerador para **`prisma-client-js`** e **apagar a linha `output`**. Escrever o **`model Doce`** (id, nome, preço).

Abrir: Cap. 5 §5.5.2 — ver o schema.prisma com um model
https://leofernandesmo.github.io/pweb2-book/chapters/05-persistencia/#552-o-prisma-schema

5.3 Criar o banco

Nunca `prisma db push`.

```
$ npx prisma migrate dev --name criar_tabelas_iniciais
```

Testar: a saída diz "in sync" e existe **`prisma/migrations`**.

5.4 src/config/database.js

Exportar uma instância de **`PrismaClient`** como **`prisma`**.

Abrir: Cap. 5 §5.5.4 — ver o database.js (PrismaClient)
https://leofernandesmo.github.io/pweb2-book/chapters/05-persistencia/#554-configurando-o-prisma-client

5.5 Conferir

```
$ npx prisma studio
```

Testar: a tabela aparece. Se der "did not initialize yet": **`npx prisma generate`**.

6 CRUD do cardápio de doces: antes de começar

6.1 Nomes

Exemplo: cardápio da Dona Lourdes. Recurso **`doces`**, model **`Doce`** (**`nome`**, **`preco`**), **`prisma.doce`**. **Tudo na raiz:** as URLs são **`/`**, **`/new`** e **`/ID/edit`** (sem prefixo **`/doces`**) e as views ficam direto em **`src/views`** (**`index.ejs`** e **`form.ejs`**). No livro o exemplo é **`usuarios`**: ao copiar, trocar **`usuario`** por **`doce`**.

6.2 Ordem

4 etapas (listar, criar, alterar, excluir), cada uma com teste. Em cada etapa: rotas, controller, service, repository e views, nessa ordem. O teste só funciona depois de todos os arquivos da etapa. Só avançar quando o teste passar.

7 Etapa A: listar

7.1 src/routes/index.routes.js (agregador)

O arquivo do hello world (4.1) **perde a rota do hello** e passa a montar o router de doces na raiz. O **`app.js`** não muda (já tem o **`app.use('/', router)`** do 4.2). As rotas do **`doces.routes.js`** são relativas (**`/`**, **`/new`**...).

```
import { Router } from 'express';
import docesRouter from './doces.routes.js';

const router = Router();

router.use('/', docesRouter);

export default router;
```

Abrir: Cap. 3 §3.5.4 — ver o agregador (routes/index.js com um router.use por recurso)
https://leofernandesmo.github.io/pweb2-book/chapters/03-express/#354-o-agregador-de-rotas

7.2 src/routes/doces.routes.js

Copiar o arquivo do livro e deixar só o que a etapa A usa. **A validação do livro sai daqui:** no nosso CRUD ela fica no service (8.3), então as rotas não usam **`regrasUsuario`** nem **`verificarValidacao`** (eles precisam do **`express-validator`** e do **`validacao.middleware.js`**, §6.7.3). Rota apontando para método do controller que ainda não existe **derruba o servidor na partida** (**`argument handler must be a function`**): nesta etapa, só o **`GET /`**. Trocar **`Usuarios...`** por **`Doces...`** e os caminhos para **`doces.repository.js`**, **`doces.service.js`** e **`doces.controller.js`**.

Apagar do arquivo do livro:

```
import { regrasUsuario, verificarValidacao } from '../middlewares/validacao.middleware.js';
import methodOverride from 'method-override';        // fica no app.js (9.5)
router.use(methodOverride('_method'));
router.get('/:id', controller.show);                  // não temos view de detalhe
// e as rotas /new, /:id/edit, post, put e delete: entram nas etapas B a D
```

Fica assim (etapa A):

```
import { Router } from 'express';
import { DocesRepository } from '../repositories/doces.repository.js';
import { DocesService } from '../services/doces.service.js';
import { DocesController } from '../controllers/doces.controller.js';

const controller = new DocesController(new DocesService(new DocesRepository()));
const router = Router();
router.get('/', controller.index);

export default router;
```

Abrir: Cap. 6 §6.8.1 — ver o arquivo de rotas completo
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#681-rotas-completas

7.3 src/controllers/doces.controller.js

Recebe o service no construtor; **`index`** busca os doces e renderiza a view **`index`**.

```
export class DocesController {
  constructor(service) {
    this.service = service;
  }
  index = async (req, res, next) => {
    try {
      const doces = await this.service.listarTodos();
      res.render('index', { doces });
    } catch (err) { next(err); }
  };
}
```

7.4 src/services/doces.service.js

Construtor recebe o repository; **`listarTodos`**. **Não copiar o `criar` do livro** (e-mail e **`bcrypt`**). Com **`AppError`**, criar **`src/utils/AppError.js`**.

Abrir: Cap. 4 §4.6.3 — ver o service (buscarPorId, atualizar, remover)
https://leofernandesmo.github.io/pweb2-book/chapters/04-arquitetura/#463-o-service-refatorado

Abrir: Cap. 3 §3.5.5 — ver a classe AppError
https://leofernandesmo.github.io/pweb2-book/chapters/03-express/#355-classe-de-erro-customizado

7.5 src/repositories/doces.repository.js

Só o **`listarTodos`** nesta etapa; usar **`prisma.doce`**. Ignorar o **`buscarPorEmail`**.

Abrir: Cap. 5 §5.7.1 — ver o repository com Prisma
https://leofernandesmo.github.io/pweb2-book/chapters/05-persistencia/#571-usuariosrepositoryprisma

7.6 src/views/index.ejs

Substitui o hello do 4.3: mesmo esqueleto (head igual); no **<body>**, a lista com **forEach** (nome e preço). Trocar **usuarios** por **doces**, ignorar o **flash**, **sem** Alterar e Excluir por enquanto.

Abrir: Cap. 6 §6.4.2 — ver a lista com forEach, link Editar e formulário de exclusão
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#642-condicionais-e-loops

7.7 Teste da etapa

Testar: abrir **http://localhost:3000/**: título e lista vazia, sem erro (o hello sumiu). Inserir um doce pelo **npx prisma studio** e recarregar: ele aparece.

8 Etapa B: criar

8.1 src/routes/doces.routes.js

Acrescentar **GET /new** e **POST /**. **No POST, sem validação na rota:** o livro põe **regrasUsuario** e **verificarValidacao('usuarios/new')** antes do controller. Apague as duas (se já apagou a **regrasUsuario**, apague também a **verificarValidacao**). A validação é o **validar** do service (8.3).

Do livro:

```
router.post('/',
  regrasUsuario,
  verificarValidacao('usuarios/new'),
  controller.create
);
```

Fica assim:

```
router.get('/new', controller.new);
router.post('/', controller.create);
```

8.2 src/controllers/doces.controller.js

Acrescentar **new** e **create**.

```
  new = (req, res) => res.render('form', { doce: null, erro: null });

  create = async (req, res) => {
    try {
      await this.service.criar(req.body);
      res.redirect('/');
    } catch (err) {
      res.status(400).render('form', { doce: req.body, erro: err.message });
    }
  };
```

8.3 src/services/doces.service.js

Acrescentar **validar** e **criar**.

```
  validar({ nome, preco }) {
    const n = String(nome ?? '').trim();
    const p = Number(String(preco ?? '').replace(',', '.'));
    if (!n) throw new Error('Nome é obrigatório');
    if (Number.isNaN(p) || p < 0) throw new Error('Preço inválido');
    return { nome: n, preco: p };
  }

  criar(dados) {
    return this.repository.criar(this.validar(dados));
  }
```

8.4 src/repositories/doces.repository.js

Acrescentar **criar**.

8.5 src/views/form.ejs

Esqueleto do 4.3; **<body>**:

```
  <h1>Novo doce</h1>
  <% if (erro) { %><p><%= erro %></p><% } %>
  <form method="POST" action="/">
    <input name="nome" value="<%= doce ? doce.nome : '' %>">
    <input name="preco" value="<%= doce ? doce.preco : '' %>">
    <button type="submit">Salvar</button>
  </form>
```

8.6 src/views/index.ejs

Acrescentar o link para **/new**.

Abrir: Cap. 6 §6.5.2 — ver o index.ejs com o botão Novo
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#652-usando-partials-nos-templates

8.7 Teste da etapa

Testar: abrir **`/new`** e cadastrar um doce com preço **`3,50`**: aparece na lista. Nome vazio: erro no formulário. **Reiniciar o servidor:** o doce continua.

9 Etapa C: alterar

9.1 src/routes/doces.routes.js

Acrescentar **`GET /:id/edit`** e **`PUT /:id`**.

9.2 src/controllers/doces.controller.js

Acrescentar **`edit`** e **`update`**.

```
  edit = async (req, res, next) => {
    try {
      const doce = await this.service.buscarPorId(Number(req.params.id));
      res.render('form', { doce, erro: null });
    } catch (err) { next(err); }
  };

  update = async (req, res) => {
    const id = Number(req.params.id);
    try {
      await this.service.atualizar(id, req.body);
      res.redirect('/');
    } catch (err) {
      res.status(400).render('form', { doce: { id, ...req.body }, erro: err.message });
    }
  };
```

9.3 src/services/doces.service.js

Acrescentar **`buscarPorId`** (erro se o id não existe) e **`atualizar`**, passando os dados por **`this.validar(dados)`**.

9.4 src/repositories/doces.repository.js

Acrescentar **`buscarPorId`** e **`atualizar`**.

9.5 src/app.js

Importar o **`method-override`** e adicionar **`app.use(methodOverride('_method'))`** **antes** do **`app.use('/', router)`**.

Abrir: Cap. 6 §6.7.5 — ver o import e o app.use do method-override
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#675-method-override-put-e-delete-em-formularios-html

9.6 src/views/form.ejs

Trocar a linha do **`<form>`**:

```
  <form method="POST" action="<%= doce && doce.id ? `/${doce.id}?_method=PUT` : '/' %>">
```

9.7 src/views/index.ejs

Acrescentar o link Alterar (**`href="/ID/edit"`**) em cada doce.

Abrir: Cap. 6 §6.4.2 — ver a lista com forEach, link Editar e formulário de exclusão
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#642-condicionais-e-loops

9.8 Teste da etapa

Testar: Alterar abre o formulário preenchido; salvar volta à lista com o valor novo. **`Cannot POST`** = falta o **`method-override`** (9.5).

10 Etapa D: excluir

10.1 src/routes/doces.routes.js

Acrescentar **`DELETE /:id`**.

10.2 src/controllers/doces.controller.js

Acrescentar **`destroy`**.

```
  destroy = async (req, res, next) => {
    try {
      await this.service.remover(Number(req.params.id));
      res.redirect('/');
    } catch (err) { next(err); }
  };
```

10.3 src/services/doces.service.js

Acrescentar **`remover`** (busca o doce e chama o repository).

10.4 src/repositories/doces.repository.js

Acrescentar **`remover`**.

10.5 src/views/index.ejs

Acrescentar o formulário Excluir em cada doce (POST com **`?_method=DELETE`**).

Abrir: Cap. 6 §6.4.2 — ver a lista com forEach, link Editar e formulário de exclusão
https://leofernandesmo.github.io/pweb2-book/chapters/06-ejs/#642-condicionais-e-loops

10.6 Teste da etapa

Testar: excluir: o doce some da lista, também depois de recarregar e de **reiniciar o servidor**. Em **`/`**, **`/new`** e **`/ID/edit`** o fundo verde aparece (aba Network: CSS e JS com status 200).

11 Prova: uma tela com 2 CRUDs relacionados

11.1 Dois CRUDs, uma tela

Repetir os passos 6 a 10 para o segundo recurso (ex.: **`categorias`**), com model, rotas, controller, service e repository próprios. **Dois CRUDs não cabem os dois na raiz** (**`/new`** e **`/:id/edit`** colidiriam): voltar ao prefixo, e os links, **`action`**s e **`redirect`** ganham ele (**`/doces/new`**, **`/categorias/ID?_method=DELETE`**...). A tela única é o **`index.ejs`**, servido por um **`GET /`** no agregador que busca as duas listas. Os **`POST`**, **`PUT`** e **`DELETE`** seguem nas rotas de cada recurso e terminam com **`res.redirect('/')`**.

No agregador (routes/index.routes.js):

```
router.get('/', homeController.index);
router.use('/doces', docesRouter);
router.use('/categorias', categoriasRouter);
```

No home.controller.js (recebe os dois services):

```
const [doces, categorias] = await Promise.all([
  this.doceService.listarTodos(),
  this.categoriaService.listarTodos(),
]);
res.render('index', { doces, categorias });
```

11.2 Relacionamento 1:N

O filho guarda a chave estrangeira. Depois de mudar o schema, rodar **`migrate dev`**. O livro cria o filho com **`connect: { id }`**; usar **`categoriaId`** direto também funciona.

```
model Categoria {
  id    Int    @id @default(autoincrement())
  nome  String
  doces Doce[]
}

model Doce {
  id          Int       @id @default(autoincrement())
  nome        String
  preco       Float
  categoriaId Int
  categoria   Categoria @relation(fields: [categoriaId], references: [id])
}
```

Abrir: Cap. 5 §5.8.1 — ver o schema 1:N (@relation e onDelete) e a consulta com include
https://leofernandesmo.github.io/pweb2-book/chapters/05-persistencia/#581-relacionamento-1n-e-n1-um-para-muitos-muitos-para-um

11.3 Listar com o relacionado

No **`findMany`** do repository do filho, **`include: { categoria: true }`**; na view, **`doce.categoria.nome`**.

Abrir: Cap. 5 §5.5.6 — ver include, connect e select em consultas com relação
https://leofernandesmo.github.io/pweb2-book/chapters/05-persistencia/#556-relacionamentos-com-prisma

11.4 Formulário do filho

Um **`select`** com as categorias. No service, converter com **`Number(categoriaId)`**: string dá erro do Prisma (**`Expected Int, provided String`**).

```
<select name="categoriaId">
  <% categorias.forEach(c => { %><option value="<%= c.id %>"><%= c.nome %></option><% }) %>
</select>
```

11.5 Excluir o pai

Excluir uma categoria que tem doces dá erro **`P2003`**. Escolher: **`onDelete: Cascade`** no **`@relation`** (apaga os doces junto) ou tratar o erro e avisar.

Abrir: Cap. 5 §5.8.3 — ver as opções de onDelete (Cascade, Restrict, SetNull)
https://leofernandesmo.github.io/pweb2-book/chapters/05-persistencia/#583-boas-praticas-com-relacionamentos

11.6 Teste da etapa

Testar: criar uma categoria e um doce escolhendo ela: a lista mostra o nome da categoria. Excluir a categoria com doces: o comportamento escolhido no 11.5.